

Rapport ML : analyse de satisfaction des messages

I - Analyse de la problématique

L'application Weeb dispose d'un formulaire de contact qui collecte des messages. Ces messages s'accumulent en base sans qu'il soit possible de savoir rapidement si les retours sont positifs ou négatifs.

Objectif : classifier automatiquement chaque message comme "satisfait" (1) ou "non satisfait" (0).

Type de problème ML : classification binaire supervisée. Entrée : texte brut du champ message. Sortie : label 0 ou 1.

Justification : les messages de contact sont courts, expriment une opinion, et existent en anglais comme en français puisque les utilisateurs de Weeb peuvent écrire dans les deux langues. La classification binaire supervisée est directement adaptée à ce besoin.

Méthodologie : découpage train/test simple avec `random_state`, entraînement via `fit`, évaluation via `score`, `accuracy`, `precision`, `recall`, `F1-score` et matrice de confusion, comparaison du score train et du score test pour détecter le surapprentissage. Une étape préalable est nécessaire avant tout entraînement : la vectorisation du texte (TF-IDF), détaillée en section "Les algorithmes testés".

II - L'origine des données utilisées et leur qualification

Origine des données

Deux sources publiques ont été combinées pour obtenir un dataset bilingue.

Dataset anglais : Sentiment Labelled Sentences

Source : UCI Machine Learning Repository.

<https://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>

Ce dataset contient 3 000 phrases courtes, réparties en 3 fichiers de 1 000 phrases chacun, chaque phrase étant étiquetée 0 (négatif) ou 1 (positif) :

Fichier	Origine des textes	Exemples	Positifs	Négatifs
amazon_cells_labelled.txt	Avis de produits Amazon	1 000	500	500

yelp_labelled.txt	Avis de restaurants Yelp	1 000	500	500
imdb_labelled.txt	Avis de films IMDB	1 000	500	500

Dataset français : Allociné

Source : dataset Allociné. <https://huggingface.co/datasets/tblard/allocine>

Ce dataset contient environ 160 000 critiques de films en français, chacune étiquetée 0 (négatif) ou 1 (positif). Un échantillon aléatoire équilibré de 3 000 lignes (1 500 positives, 1 500 négatives, `random_state=42` pour la reproductibilité) a été prélevé, afin d'obtenir un volume comparable au dataset anglais.

Pourquoi ces sources : ce sont des jeux de données publics, largement utilisés dans la littérature académique, déjà annotés par des humains, gratuits. Leur combinaison permet d'entraîner un seul modèle capable de traiter aussi bien l'anglais que le français.

Qualification des données

Le script `ml/prepare_data.py` affiche à l'exécution des statistiques de contrôle sur le dataset fusionné, avant nettoyage :

- **Total avant nettoyage :** 6 000 lignes (3 000 anglais, 3 000 français), équilibrées à 50% positif et 50% négatif
- **Doublons trouvés :** 21 phrases strictement identiques entre les sources
- **Valeurs manquantes :** aucune, sur les colonnes `text` et `label`
- **Longueur moyenne des phrases :** environ 12 tokens en anglais contre environ 91 tokens en français.
- **Équilibre des classes :** confirmé à 50/50, aucun ré échantillonnage supplémentaire nécessaire

III - Procédures d'importation, de nettoyage et de structuration des données

Le script `ml/prepare_data.py` réalise toute la préparation des données en une seule commande : `python prepare_data.py`.

1. **Import des fichiers anglais :** les 3 fichiers `.txt` sont au format TSV (texte et label séparés par une tabulation), lus avec `pandas.read_csv`. L'option `quoting=csv.QUOTE_NONE` a été nécessaire car certaines critiques IMDB contiennent des guillemets non appariés.
2. **Import du fichier français :** `allocine_sample.csv`, déjà échantillonné et équilibré au préalable, encodé en UTF-8.

3. **Structuration** : les deux sources sont assemblées dans un seul tableau à 4 colonnes : text, label, source (amazon, yelp, imdb ou allocine) et lang (en ou fr), pour garder la traçabilité de chaque ligne.
4. **Nettoyage** : suppression des doublons (phrases strictement identiques entre les sources) et suppression des lignes avec des valeurs manquantes sur text ou label.
5. **Sauvegarde** : le résultat final, 5 979 lignes après nettoyage, est écrit dans ml/data/sentiment_data.csv.

Reproductibilité : les fichiers sources bruts (les 3 .txt et allocine_sample.csv) sont conservés. Le fichier fusionné sentiment_data.csv se régénère à l'identique en relançant python prepare_data.py.

IV - Algorithmes testés et le choix du modèle de machine learning

La vectorisation TF-IDF

Avant tout entraînement, chaque phrase doit être transformée en nombres, car aucun modèle scikit-learn ne peut recevoir du texte brut en entrée. Le TfidfVectorizer de scikit-learn donne à chaque mot un score selon sa fréquence dans la phrase, en le pondérant à la baisse s'il est très fréquent dans tout le corpus.

Approche retenue

Un pipeline scikit-learn associant le TfidfVectorizer à un modèle de classification, entraîné en apprentissage supervisé sur les données décrites plus haut.

Algorithmes testés

Deux algorithmes ont été entraînés et comparés dans ml/train.py, sur le même pipeline de vectorisation et le même découpage train/test (80% / 20%, random_state=42, sans validation croisée) :

Modèle	Rôle dans la comparaison
Logistic Regression	Modèle linéaire, interprétable, bon point de départ pour la classification
Decision Tree (arbre de décision)	Modèle non linéaire, permet de comparer à une approche différente

Résultats obtenus

Modèle	Score train	Score test	Accuracy	Precision	Recall	F1
Logistic Regression	0.940	0.844	0.844	0.832	0.852	0.842
Decision Tree	1.000	0.692	0.692	0.680	0.692	0.686

Détail de l'accuracy par langue sur le jeu de test (601 exemples anglais, 595 exemples français) :

Modèle	Anglais	Français
Logistic Regression	0.837	0.852
Decision Tree	0.694	0.691

Détection du surapprentissage

Comparer le score sur le jeu d'entraînement et le score sur le jeu de test permet de détecter le surapprentissage.

Le **Decision Tree** en est un exemple concret : score train de 1.000 contre un score test de seulement 0.692, un écart de 0.308. C'est le phénomène typique du surapprentissage : un modèle très performant sur des données qu'il connaît déjà mais qui généralise mal sur de nouvelles données.

La **Logistic Regression** montre un écart bien plus faible entre score train (0.940) et score test (0.844), soit 0.096, signe d'un modèle qui généralise correctement plutôt que d'avoir mémorisé les données d'entraînement.

Modèle retenu

Le modèle retenu est **Logistic Regression**, sélectionné sur le F1-score obtenu sur le jeu de test (0.842 contre 0.686 pour Decision Tree). C'est aussi le modèle qui présente le moins d'écart entre score train et score test, donc le moins de risque de surapprentissage.

Les deux langues sont bien traitées par ce modèle : accuracy de 0.837 en anglais et 0.852 en français.

V - Les métriques utilisées pour évaluer le modèle

Pour chaque modèle testé, les métriques suivantes ont été calculées :

- **Score train et score test**, pour détecter un éventuel surapprentissage
- **Accuracy** sur le jeu de test, 20% des données, jamais vues pendant l'entraînement
- **Precision et recall**, calculés avec `precision_score` et `recall_score` de scikit-learn
- **F1-score**, calculé avec `f1_score`, utilisé comme critère de sélection du meilleur modèle
- **Matrice de confusion**, calculée avec `confusion_matrix`, pour visualiser le détail des erreurs (faux positifs et faux négatifs)

Rapport détaillé pour le modèle retenu, Logistic Regression :

Classe	Precision	Recall	F1-score	Support
0 (non satisfait)	0.86	0.84	0.85	615

1 (satisfait)	0.83	0.85	0.84	581
Accuracy globale			0.84	1 196

Matrice de confusion associée :

	Prédit 0	Prédit 1
Réel 0	515	100
Réel 1	86	495

Sur 1 196 messages de test, 615 étaient réellement négatifs et 581 réellement positifs. Le modèle a correctement classé 515 des 615 négatifs et 495 des 581 positifs.

VI - Recommandations

Limitations identifiées

- Seules deux langues sont couvertes.
- Les phrases françaises sont en moyenne bien plus longues que les phrases anglaises.
- Les données d'entraînement ont un style différent des messages de contact réels de Weeb. Les performances mesurées ici pourraient être légèrement supérieures à celles obtenues en conditions réelles.
- 5 979 exemples sont suffisants pour ce projet, mais resteraient limités pour une mise en production à grande échelle.
- Les labels sont binaires uniquement, aucune nuance de type "neutre" ou "très mécontent" n'est possible.

Pistes d'amélioration

- Étendre le dataset à d'autres langues (espagnol, allemand).
- Annoter manuellement un échantillon de vrais messages de contact Weeb (français et anglais) pour évaluer, voire ré-entraîner, le modèle sur le domaine cible réel plutôt que sur des avis de produits ou de films.
- Explorer d'autres modèles ou méthodes d'évaluation spécialisés pour le texte pour affiner davantage les performances.